

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

Projektarbeit T3_3100

des Studiengangs **Informatik IT-Security** an der
Dualen Hochschule Baden-Württemberg Ravensburg
Campus Friedrichshafen

von **Leon Luca Eltrich**

Abgabedatum: **08.09.2025**

Bearbeitungszeitraum:	10 Wochen
Matrikelnummer, Kurs:	4276625, TIS23
Dualer Partner:	doubleSlash Net-Business GmbH, Friedrichshafen
Betreuer*in des Dualen Partners:	- - -
Gutachter*in der Dualen Hochschule:	Prof. Dr. Axel Hoff

Eigenständigkeitserklärung

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema:

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum

Unterschrift

Abbreviations

LLM Large Language Model

Inhaltsverzeichnis

1	Introduction	1
1.1	Background and Motivation	1
1.2	The Problem	1
1.3	Proposed Solution: A Local-First Approach	2
1.4	Research Questions	2
1.5	Expected Contributions	2
2	Background	4
3	Methodology	5
4	Literature Review	6
5	Systems Design And Implementation	7
5.1	Design	7
5.1.1	Requirements	7
5.1.2	Business Cases?	7
5.2	Implementation	7
6	Evaluation	8
6.1	Technical Benchmarking	8
6.2	User Study	8
7	Results	9
8	Discussion	10
8.1	Validity Concerns	10
8.2	Answers To The Research Questions	10
9	Conclusion And Future Work	11

1 Introduction

1.1 Background and Motivation

For decades, the Command Line Interface (CLI) has served as a fundamental and powerful mechanism for navigating and managing computer systems. Command-line applications remain ubiquitous across software development, system administration, data analysis, and scientific computing. They offer unmatched efficiency, scriptability, and granular control over system resources.

However, despite its powerful features and long-standing existence, the command line continues to present a challenge, especially to novice users. The interface relies heavily on strict syntactical rules, cryptic abbreviations, and inconsistent command option flags across different utilities. While newcomers might struggle to interpret the highly technical descriptions found in traditional man pages or help manuals, the cognitive load affects seasoned users as well. Applications frequently possess an excessive number of options, flags, and arguments, making it difficult to utilize tools to their full potential without constant reference to external documentation.

1.2 The Problem

The traditional methods for bridging this knowledge gap, such as manual page consultation, web searches, or community forums, often require significant context switching, which disrupts the user's workflow. Recently, cloud-based Large Language Models (LLMs) such as ChatGPT or Gemini have emerged as popular alternatives for providing immediate command-line guidance.

While they offer highly intuitive natural language interaction, they suffer from several critical limitations in the context of system administration and software development:

- **Privacy and Security Risks:** Relying on cloud APIs requires transmitting user queries – which may contain sensitive system paths, file names, or proprietary tool names – to external servers.
- **Networking Constraints:** Cloud models are inherently unavailable in restricted, air-gapped, or offline environments, which are common in enterprise server management and secure development setups.
- **Domain-Specific Knowledge Gap:** General-purpose models often lack knowledge of internal, closed-source, or highly niche CLI applications specific to a particular organization's workflow.
- **Resource Usage:** While open-source LLMs exist, they tend to be quite hardware and resource heavy, which limits their feasibility.

1.3 Proposed Solution: A Local-First Approach

This thesis aims at addressing the challenges outlined in section 1.2 by exploring a local-first, privacy-preserving and extensible command option retrieval system. The core objective is to design, implement and evaluate a system that interprets natural-language prompts and accurately retrieves the correct command-line options without relying on external services.

By utilizing natural language processing approaches such as embeddings and lightweight language models, the proposed system will map user-generated descriptions of what they are trying to do (e.g. "List all files in this directory, including the ones not shown.") to the command-line options required for the task. Crucially, this proof-of-concept is designed to operate strictly offline, with a focus on acceptable response times and hardware usage, utilizing only standard CPU hardware.

1.4 Research Questions

To systematically evaluate the feasibility and performance of the proposed local-first approach, this thesis seeks to address several core research questions. These questions are designed to bridge the gap between technical optimization and human-centric usability, focusing specifically on the constraints of offline, CPU-bound environments.

The study is guided by the following specific research questions:

- **RQ1:** Which local NLP techniques can effectively map natural language descriptions to command-line options with limited hardware resources available?
- **RQ2:** What trade-offs exist between accuracy, resource usage and extensibility in such a system?
- **RQ3:** How can the system be extended with new commands and options with minimal overhead?

1.5 Expected Contributions

This thesis provides both a practical tool and new scientific insights into how natural language can improve command-line interactions. The primary contributions include:

First, this work delivers a functional proof-of-concept system that proves it is possible to retrieve command options locally and privately. Second, it provides a detailed analysis of the technical trade-offs – balancing accuracy, speed, and memory usage – when running NLP models on standard CPU hardware.

Additionally, the thesis introduces an extensible architecture that allows developers to easily add natural language support to other CLI tools. Finally, the research offers an understanding

of the perceived effectiveness of the tool as experienced by users. Together, these results offer a practical path toward making the command line more accessible without compromising user privacy.

2 Background

3 Methodology

4 Literature Review

5 Systems Design And Implementation

5.1 Design

Database: - Tabelle für Tool beschreibungen - Tool name (ID) - Tool Beschreibung (TEXT) - Tool Beschreibung Vector (BLOB) - Tabelle für tool option Beschreibungen - option (ID) - Tool Name (ID) - Tool Option beschreibung (TEXT) - Tool Option Beschreibung Vector (BLOB)

Pipeline Design

5.1.1 Requirements

- Niedrige Latenz - Ram <1GB - 85 Prozent akkurat - Schneller Startup - Keine Internetanbindung nötig (Läuft lokal) - (Einfach) Erweiterbar - Embeddings der Tools müssen gespeichert werden dass sie nicht die ganze zeit berechnet werden müssen - Die rohen beschreibungen müssen gespeichert werden - Daten müssen gespeichert werden ohne anbindung an ressourcen intensiven Datenbank server - Es muss eine Validierung der Command syntax geben - Das Embedding modell muss austauschbar sein um in zukunft ein anderes verwenden zu können - Es soll eine kompiliere Sprache genutzt werden um schnelle Startup zeiten und portabilität zu gewährleisten

5.1.2 Business Cases?

user erfragt tool das kein options erfordert user erfragt tool das eine option erfordert user erfragt tool das mehrere options erfordert

entwickler möchte das modell austauschen entwickler möchten ihre anwendung registrieren

5.2 Implementation

- Rust

tar, mv, cp,

6 Evaluation

6.1 Technical Benchmarking

6.2 User Study

7 Results

8 Discussion

8.1 Validity Concerns

8.2 Answers To The Research Questions

9 Conclusion And Future Work

Abbildungsverzeichnis

Tabellenverzeichnis

1	Eingesetzte KI-Werkzeuge und Beschreibungen deren Verwendung	14
---	--	----

KI-Verzeichnis

Werkzeug	Beschreibung der Nutzung
ChatGPT	Der Einsatz erfolgte in der gesamten Arbeit zur sprachlichen Gestaltung. Konkret diente es dazu, vorhandene Texte stilistisch an den wissenschaftlichen Schreibstil anzupassen und präzise Formulierungen zu entwickeln. Außerdem wurde wie in Kapitel ?? bzw. Kapitel ?? beschrieben, die Funktion <i>ChatGPT Deep Research</i> für das Identifizieren relevanter Quellen genutzt. Limitationen wurden in besagten Kapiteln reflektiert. Darüber hinaus kam ChatGPT zur Klärung fachlicher Verständnisfragen im Verlauf der Recherche und Umsetzung des Projektes zum Einsatz.
Perplexity	Der Einsatz erfolgte wie in den Kapiteln ?? und ?? beschrieben, um relevante Quellen zu identifizieren. Wie bei <i>ChatGPT Deep Research</i> , wurde der Einsatz in den Kapiteln kritisch hinterfragt und Limitationen identifiziert.

Tabelle 1: Eingesetzte KI-Werkzeuge und Beschreibungen deren Verwendung